

PLATFORM ENGINEERING CHALLENGE

PR Concierge

A webhook-driven service that turns pull request activity into a fast, explainable risk signal — with an AWS footprint small enough to review in one sitting.

HONO + LAMBDA

OPENTOFU-MANAGED AWS

DETERMINISTIC RULES FIRST

ARCHITECTURE STANCE

Keep the request path short and the ops story obvious.

The design favors burst-friendly compute, readable infrastructure, and clear reviewer talking points over a larger always-on platform shape.

INGRESS

GitHub → API Gateway → Lambda

A clean event path for short-lived webhook work.

What the service does

One inbound webhook becomes one fast platform signal: verify the request, inspect the pull request, score risk, and save a result that a human can act on.

RECEIVE

Accept GitHub pull request events

The HTTP surface stays deliberately small: `GET /health` and `POST /webhooks/github`.

VERIFY

Reject bad or unsigned requests early

Webhook signature validation happens before the service spends time on GitHub API work.

ANALYZE

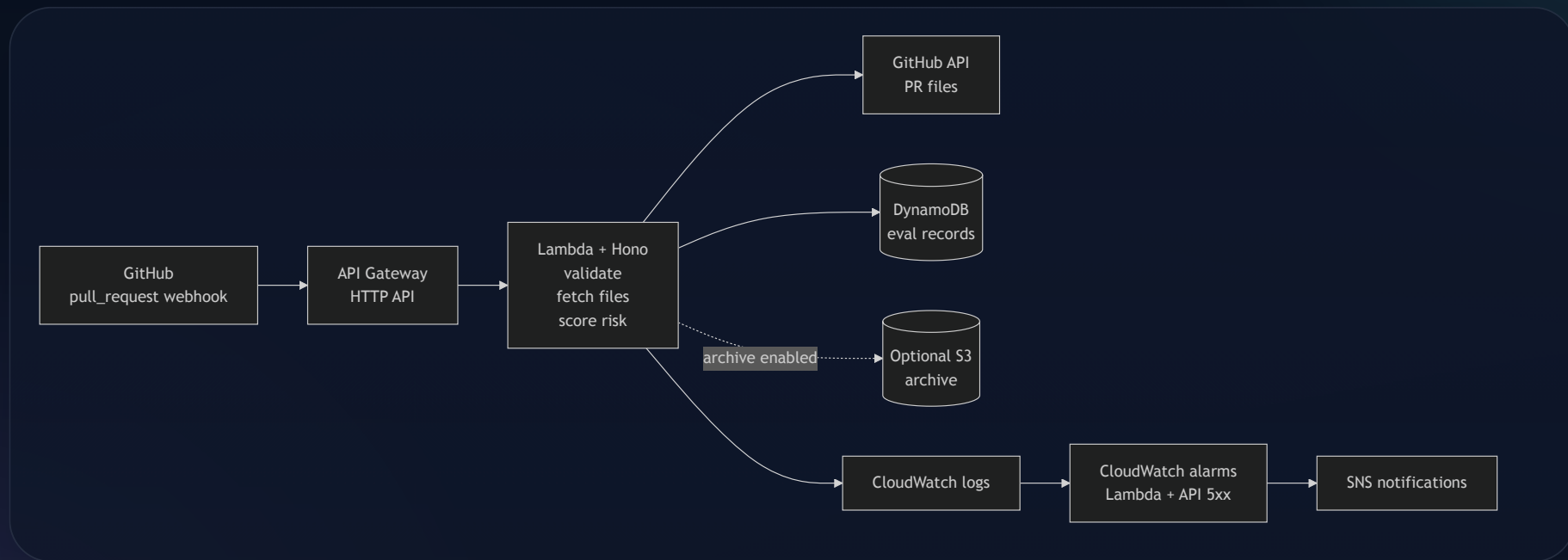
Apply deterministic checks and risk rules

Branch naming, optional labels, and changed-file risk scoring are explainable and easy to test.

PERSIST

Architecture at a glance

The runtime path stays short. Storage and operational telemetry branch cleanly from the Lambda handler instead of hiding inside later follow-up work.



Request flow and delivery path

The runtime behavior and the operator workflow are both explicit. That keeps the interview story crisp and the repo easy to navigate.

RUNTIME FLOW

1 Receive and verify

The request enters through `POST /webhooks/github`, and signature checking rejects bad inputs before deeper processing.

2 Fetch and classify

The service reads changed files, applies deterministic rules, and derives a risk level.

3 Persist and respond

The evaluation record and structured logs make the outcome visible immediately.

DELIVERY FLOW

1 Verify locally

`npm run test` proves the Lambda handler and local integration path.

2 Deploy through CI

`.github/workflows/deploy.yml` builds, tests, validates OpenTofu, and deploys.

3 Opt into self-dogfooding

`scripts/configure-self-webhook.sh` is the explicit step that turns this repo into its own webhook source.

Observability and scope control

Modern decks look cleaner when each slide makes one argument. This one is simple: the MVP is operationally aware, but it refuses to fake completeness.

ALREADY IMPLEMENTED

LOGGING

Structured JSON output

Request and evaluation context are available without hunting through plain text logs.

ALARMING

CloudWatch metric alarms

The deployed stack watches Lambda errors and API Gateway 5xx signals.

NOTIFICATIONS

SNS fan-out

Operational noise has a single handoff point instead of hidden one-off integrations.

VERIFICATION

Key design decision

API Gateway plus Lambda is the MVP shape because it keeps the product, deployment, and demo path aligned around one bursty request model.

CHOSEN FOR THE MVP

API Gateway + Lambda

- **Right-sized for webhook traffic**

The service wakes up for short-lived work instead of paying for idle containers.

- **Faster to explain and review**

The AWS footprint stays small enough for a technical interviewer to reason about quickly.

- **Easy to evolve later**

If the workload changes, the boundary around request handling is still clear enough to migrate.

ALTERNATIVES CONSIDERED

- **ECS Fargate**

Better for longer-lived runtime control, but heavier than the current traffic pattern needs.

- **Broader multi-service footprint**

Would add delivery surface area before the webhook workflow itself had earned it.

READ NEXT

The full prose version of this tradeoff lives in `DECISIONS.md`, so the reviewer can switch from visual summary to rationale without leaving the repo.

Start with the smallest proof, then step outward into persistence, logs, and infrastructure.

01

Health check

Show `GET /health` first to prove the runtime surface is alive.

02

Signed webhook

Send a realistic pull request event through the verified webhook path.

03

Saved evaluation

Inspect the resulting evaluation record and the deterministic summary.

04

Ops signals

Close on logs, alarms, and deploy artifacts to prove operational maturity.